

Webscraping and Social Media Scraping

Basic HTML navigation

Dr Maciej Świtła and Ewa Weychert

About me ;)



- **PhD candidate at Faculty of Economic Sciences**
- **Research focus on** machine learning and programming, text mining, demography especially fertility and uncertainties
- **Conducting courses at Data Science**
 - 1 Programming in Python
 - 2 Webscraping and Social Media Scraping
 - 3 **Machine Learning 2:** predictive models, deep learning, neural network
 - 4 **Master seminar *Narrative Economics*: from Text Mining to Search for Meaning**
- Researcher at **LabFam**
- Researcher at University of Florence
- Email: e.weychert@uw.edu.pl

Plan for Today

- Introduction to jupyter notebook and python
- Introduction to html structures, parsing and web inspection
- Simple example of scraping with BeautifulSoup and requests
- Students' Exercises

Web scrapping definition

- Web scrapers are excellent at gathering and processing large amounts of data.
- Steps in web scraping projects:
 - Retrieving HTML data from a domain name (how the browser interprets HTML, CSS, and JavaScript)
 - Parsing that data for target information
 - Storing the information
 - Optionally moving to another page to repeat the process
- Rather than viewing one page at a time through the narrow window of a monitor, you can view databases spanning thousands or even millions of pages at once.
- Today we will start with the basics:
 - Sending a GET request to a web server (requests library)
 - Reading the HTML output from that page (BeautifulSoup library)
 - Doing some simple data extraction

What Skills Do You Need?

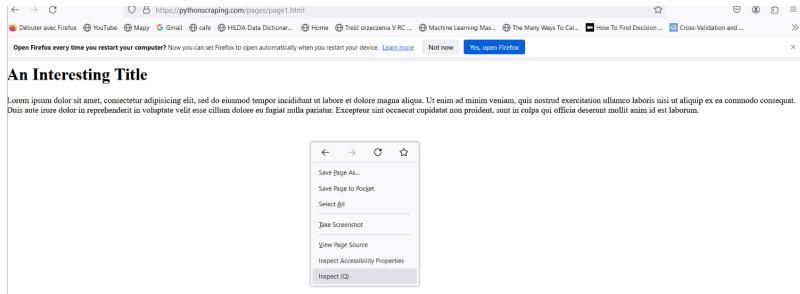
Aim of the Course

- **Web scraping** is the process of extracting data from a website.
- Although only basic knowledge of Python is required to start, web scraping can become more complex because it goes beyond Python.
- It also involves learning:
 - Basics of HTML
 - XPath
 - Libraries such as BeautifulSoup, Selenium, or Scrapy
- A typical step in web scraping is inspecting an element in a website to obtain the HTML code behind it.
- This is why learning the basics of HTML is essential before using any web scraping library.
- You should also be familiar with:
 - pandas
 - if statements
 - Loops in Python

Inspect a webpage

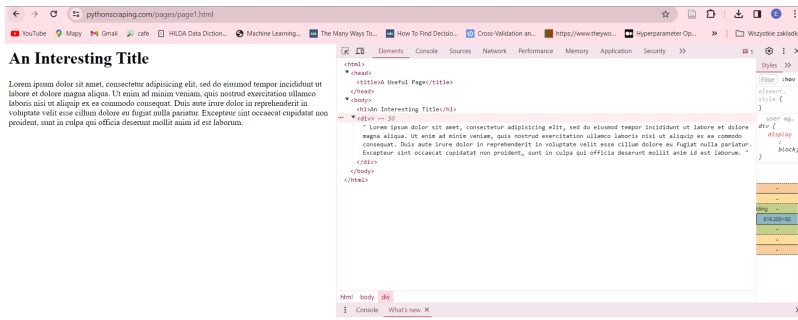
Go to this link

<http://pythonscraping.com/pages/page1.html>



Click on the right mouse button and choose **Inspect**

What skills you need?



You will see a html code. How to extract it and how to save it without clicking using Python? **HTML Parsing**

HTML Parsing

HTML Parsing the process of analyzing an HTML document and extracting its structural components, such as tags, attributes, and content, to create a structured representation of the document. How to do it in Python?

BeautifulSoup library — why this name?

“Beautiful Soup, so rich and green, Waiting in a hot tureen! Who for such dainties would not stoop? Soup of the evening, beautiful Soup!”

- The **BeautifulSoup** library was named after a **Lewis Carroll** poem of the same name in ***Alice's Adventures in Wonderland***.

In the story, the poem is sung by the **Mock Turtle** (a reference to the Victorian dish Mock Turtle Soup).

- Like its Wonderland namesake, **BeautifulSoup** tries to make sense of the nonsensical.
- It helps format and organize messy HTML by fixing broken markup and presenting easily-traversable Python objects representing structured data.



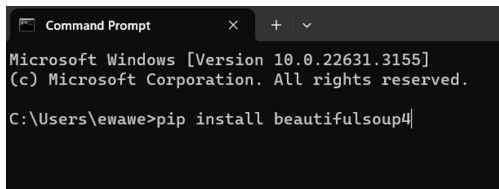
BeautifulSoup

Install in cmd (Command Prompt)

The **BeautifulSoup** and **requests** library is not a default Python library. **It must be installed!**

```
pip install beautifulsoup4
```

```
pip install requests
```

A screenshot of a Windows Command Prompt window. The title bar says "Command Prompt". The text inside the window reads: "Microsoft Windows [Version 10.0.22631.3155] (c) Microsoft Corporation. All rights reserved. C:\Users\ewawe>pip install beautifulsoup4|".

```
Command Prompt
Microsoft Windows [Version 10.0.22631.3155]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ewawe>pip install beautifulsoup4|
```

<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

Request

```
# 1. our adres url
url = "http://www.pythonscraping.com/exercises/exercise1.html"
```

```
# 2. get the content of webpage
response = requests.get(url)
```

```
# 3. extract text
html = response.text
```

```
html
```

The Python *requests* module has built-in methods for making **HTTP** requests with **GET**, **POST** or **PUT**. It uses a request-response protocol between a client and a server.

```
'<html>\n<head>\n<title>A Useful Page</title>\n</head>\n<body>\n<h1>An Interesting Title</h1>\n<div>\nLorem ipsum dolor sit ame  
t, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, qu  
is nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in volu  
ptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui offi  
cia deserunt mollit anim id est laborum.\n</div>\n</body>\n</html>\n'
```

GET method This method retrieves information from a specified server by sending a request with encoded user information appended to the URL.

BeautifulSoup

Prase html

```
soup = BeautifulSoup(html, 'html.parser')
soup
```

```
<html>
<head>
<title>A Useful Page</title>
</head>
<body>
<h1>An Interesting Title</h1>
<div>
Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqu
a. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute i
rure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupida
tat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
</div>
</body>
</html>
```

Steps for now

- Requests library (use get to get the information from url adress)
- BeautifulSoup library (parse html to later on work with it and extract necessary information)



BeautifulSoup

Python notation

1. Download libraries/modules

```
import requests  
from bs4 import BeautifulSoup  
import pandas as pd
```

2. Using modules/libraries

```
requests.get(url)
```

Name of the library

Function from this library after the dot

Pandas notation

Name of the library

Function from this library after the dot

Pandas, object
So, we can use
pandas, function ☺

```
In [9]: df2 = pd.DataFrame(  
...:     {  
...:         "A": 1.0,  
...:         "B": pd.Timestamp("20130102"),  
...:         "C": pd.Series(1, index=list(range(4)), dtype="float32"),  
...:         "D": np.array([3] * 4, dtype="int32"),  
...:         "E": pd.Categorical(["test", "train", "test", "train"]),  
...:         "F": "foo",  
...:     })  
...:
```

```
In [10]: df2
```

```
Out[10]:
```

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo
1	1.0	2013-01-02	1.0	3	train	foo
2	1.0	2013-01-02	1.0	3	test	foo
3	1.0	2013-01-02	1.0	3	train	foo

```
In [11]: df2.dtypes  
Out[11]:
```

A	float64
B	datetime64[s]
C	float32
D	int32
E	category
F	object
dtype:	object

Function
from this
library after
the dot

Advanced HTML Parsing

Up to this point, we took a quick look at:

- installing and running BeautifulSoup
- selecting objects one at a time.
- Now, we'll discuss searching for tags by attributes, working with lists of tags, and parse tree navigation

Basic HTML

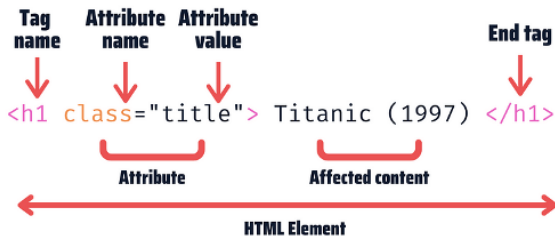
As in the example before, we are importing the `urlopen` library and calling `html.read()` in order to get the HTML content of the page. This HTML content is then transformed into a BeautifulSoup object, with the following structure:

- **html** → `<html><head>...</head><body>...</body></html>`
- **head** → `<head><title>A Useful Page</title></head>`
 - **title** → `<title>A Useful Page</title>`
- **body** → `<body><h1>An Int...</h1><div>Lorem ip...</div></body>`
 - **h1** → `<h1>An Interesting Title</h1>`
 - **div** → `<div>Lorem Ipsum dolor...</div>`

Note that the `<h1>` tag that we extracted from the page was nested two layers deep into our BeautifulSoup object structure (`html` → `body` → `h1`). However, when we actually fetched it from the object, we called the `h1` tag directly:

```
bsObj.h1
```

Tags and Attributes: What are they?



Some tags might look like this:
`<span class="green"></span>`

`<span class="red"></span>`

[Source of material and image](#)

Let's see each element.

First, the tag name is set to h1. This gives the word Titanic (1997) the largest heading size.

Some other common tags are a, p, and div.

Then we have the **attribute name** set to class, one of the most common attributes.

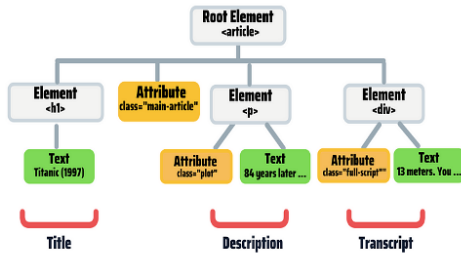
The last element is the **content**.

Web scrapers can separate elements based on their class.

For example, using BeautifulSoup you might grab all red text but ignore green text.

CSS relies on identifying attributes to style sites properly, so class and id attributes are common on modern websites.

The tree structure of the HTML



In the tree above, each rectangle represents a node.

- Gray rectangles represent **element nodes**
- Yellow rectangles represent **attribute nodes**
- Green rectangles represent **text nodes**

Also, the tree shows hierarchical relationships between nodes (parent, child, and sibling nodes).

The **root node** is the top node. In this example, `<article>` is the root.

Every node has exactly one **parent**, except the root.

An element node can have zero, one, or several **children**, but attributes and text nodes have no children.

`<p>` has two child nodes, but no child elements.

Siblings are nodes with the same parent (e.g., `h1`, `p`, and `div`). A node's children and its children's children are called its **descendants**. A node's parent and its parent's parent are called its **ancestors**.

In web scraping, locating children and parent nodes is useful when you cannot find an element but can access its parent or child.

When to get_text() and When to Preserve Tags?

- `get_text()` strips all tags from the document and returns a string containing only the text.
- If you are working with a large block of text containing hyperlinks, paragraphs, and tags, all tags will be removed and you will be left with a **tagless block of text**.
- It is usually easier to search within a **BeautifulSoup object** than inside a plain block of text.
- Calling `get_text()` should usually be the **last step**, right before printing, storing, or analyzing the final output.
- In general, try to preserve the tag structure as long as possible.

Prase htmlusing beautiful soup

```
soup = BeautifulSoup(html, 'html.parser')
soup
<html>
<head>
<title>A Useful Page</title>
</head>
<body>
<h1>An Interesting Title</h1>
<div>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
</div>
</body>
</html>
```

Print text - get_text()

```
print(soup.get_text())
```

A Useful Page

An Interesting Title

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

findAll() with BeautifulSoup

The text argument is unusual in that it matches based on the text content of the tags, rather than properties of the tags themselves. For instance, if we want to find the number of times “the prince” was surrounded by tags on the example page, we could replace our .findAll() function in the previous example with the following lines

```
nameList = bsObj.findAll(text="the prince")
print(len(nameList))
```

findAll() with BeautifulSoup

- In all likelihood, 95% of the time you will find yourself only needing to use the first two arguments: **tag** and **attributes**. However, let's take a look at all of the arguments in greater

```
findAll(tag, attributes, recursive, text, limit, keywords)  
find(tag, attributes, recursive, text, keywords)
```

- For example, the following will return a list of all the header tags in a document

```
.findAll({"h1", "h2", "h3", "h4", "h5", "h6"})
```

- The attributes argument takes a Python dictionary of attributes and matches tags that contain any one of those attributes. For example, the following function would return both the green and red span tags in the HTML document:

```
.findAll("span", {"class": "green", "class": "red"})
```

Gordian Knot – to solve a difficult problem



There are several ways in web scraping to achieve the same result.

Maybe it's buried 20 tags deep in messy HTML with no helpful tags or attributes.

Is this code readable? Easy to understand?

Gordian Knot

```
bsObj.findAll("table")[4].findAll("tr")[2].find("td").findAll("div")[1].find("a")
```


Next time: regex

References

- Mitchell, R. (2024). Web scraping with python. " O'Reilly Media, Inc.". You can download free pdf of this book from:
<https://github.com/boydfd/books/blob/master/seeing/underway/OReilly.Web.Scrapping.pdf>
- <https://medium.com/geekculture/web-scraping-cheat-sheet-2021-python-for-web-scraping-cad1540ce21c>
- Cheatsheet <https://cheatography.com/wangmz/cheat-sheets/beautiful-soup/pdf/?last=1526528008>

Mini-Exercise 1: First Successful Scrape (3 lines)

Goal: print the page **title text**.

Starter code (fill the blanks):

```
url = "http://pythonscraping.com/pages/page1.html"  
html = requests.get(url).text  
soup = BeautifulSoup(html, "html.parser")  
print("Your steps!")
```

Hints: try `soup.h1`, `soup.title`, `.get_text()`.

Mini-Exercise 2: Tag vs Text

Goal: see the difference between keeping tags and removing tags.

- 1 Print one element **with tags**.
- 2 Print the same element **as text only**.

```
print("WITH TAGS:", h1)
```

Mini-Exercise 3: Treasure Hunt with findAll()

Goal: collect all paragraphs on the page from this page:<http://pythonscraping.com/pages/warandpeace.html>

- 1 Use `findAll("p")` to get a list of paragraphs.
- 2 Print how many you found.
- 3 Print only the first 2 paragraph texts.

Mini-Exercise 4: Extract All Links from a Website

For given website: **Website:** <https://www.wne.uw.edu.pl>

1. Download the webpage

- Use `requests.get()` to retrieve the HTML
- Store the HTML content
- Create a BeautifulSoup object

2. Extract all <a> tags

- Use `soup.findAll("a")`
- Count total links
- Print the number

3. Print link text and URL

- Extract visible text
- Extract href attribute
- If missing, print "NO HREF"
- Format: Visible Text -> URL

Mini-Exercise 5: Extract and Download Image URLs

For given website: **Website:** `https://www.wne.uw.edu.pl/`

1. Download and Parse

- Use `requests.get()` (with timeout)
- Create a BeautifulSoup object

2. Extract Images

- Use `soup.find_all("img")`
- Count and print total `<img>` tags

3. Process URLs

- Get src, skip missing values
- Convert using `urljoin()`
- Remove duplicates, print first 15

4. Save Images

- Create folder (`os.makedirs()`)
- Download images and save locally

Tip (HTML example): ``